Python プログラミング入門
解答と解説

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

第1部 py ファイルの作成と実行 ..... - 1 -
第2部 変数・データ型・演算 ..... - 3 -
第3部 コレクション ..... - 13 -
第4部 条件分岐 ..... - 20 -
第5部 繰り返し ..... - 29 -
第6部 関数 ..... - 38 -
第7部 例外処理 ..... - 46 -
第8部 文字列の操作 ..... - 49 -

※各問のファイル名は、「ex＋問題番号」とすること。

例：問1-1 → ファイル名は「ex1_1」

（ファイル名から拡張子の表示は省略している）

第 1 部 py ファイルの作成と実行

問 1-1

自分の名前を標準出力で画面に表示するプログラムを作成し実行せよ。

解答例（ソースコード）

```
print("ウチダ タロウ")
```

解説

print 関数を使うと標準出力で画面に文字を表示できます。日本語を書くときは全角モードに切り替えますが、半角モードに戻すのを忘れないように気を付けてください。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 1-2

実行例のように、おはようございます、こんにちは、こんばんは、と表示するプログラムを作成し実行せよ。

解答例（ソースコード）

```
print("おはようございます")
print("こんにちは")
print("こんばんは")
```

解説

Python は基本的に一行に書く命令は一つです。改行を挟んで print 関数を 3 つ書きましょう。print 関数は、指定された文字列を表示した後に文末で自動的に改行してくれます。

第 2 部 変数・データ型・演算

問 2-1

name という変数に自分の名前を代入し、その変数を表示するプログラムを作成せよ。

解答例（ソースコード）

```
name = "ウチダタロウ"
print(name)
```

解説

変数名 = 右辺、と書くことで右辺の内容を変数に代入（定義）できます。定義した変数は、変数名を書くことで、そのプログラム中で利用できます。利用できない変数名もあるので、よく覚えていない場合は資料をチェックしてください。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 2-2

pi という変数に 3 を代入して pi を表示し、その後、変数 pi を 3.14 で上書きし、pi を表示するプログラムを作成せよ。

解答例（ソースコード）

```
pi = 3
print(pi)
pi = 3.14
print(pi)
```

解説

一度定義した変数名に値を代入すると、もともと代入されていた値を上書きすることができます。異なる型のオブジェクトで上書きすることもできます。

問2-3

変数 a に 5、変数 b に 3 を代入し、a と b の積を表示するプログラムを作成せよ。

解答例（ソースコード）

```
a = 5
b = 3
print(a * b)
```

解説

*演算子を使うと積を計算できます。計算の結果として新しいオブジェクトが生成されるので、それを print で表示しましょう。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問2-4

1つ100円のリンゴを3つ、1つ80円のみかんを5つ買ったときの合計金額を表示するプログラムを作成せよ。
ただし、ソースコードは1行のみにすること。

解答例（ソースコード）

```
print(100*3 + 80*5)
```

解説

( )を使えば計算の優先順位を変えられます。ソースコードの行数に制限がなければ以下のよう解答でも構いません。

```
apple = 100 * 3
orange = 80 * 5
sum_val = apple + orange
print(sum_val)
```

問 2-5

name という変数に自分の名前を代入し、その変数を表示するプログラムを作成せよ。ただし、name への代入は input 関数を用いて与え、ソースコードに直接自分の名前は書かないこと。

解答例（ソースコード）

```
name = input("名前を入力してください：")
print(name)
```

解説

input を書くと、プログラムはキーボードからの入力待ちになります。括弧の中身は、入力前に表示されるメッセージです。これがなくても動作しますが、プログラムの利用者にとって分かりにくいので何かメッセージを書くようにしましょう。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 2-6

input 関数を用いて名前を入力すると以下のようなメッセージを表示するプログラムを作成せよ。

解答例（ソースコード）

```
name = input("名前を入力してください：")
print("こんにちは　" + name + "さん" )
```

解説

print 関数を二つ書くと改行が入ってしまいます。＋演算子を用いて文字列を結合すれば、改行することなく一行で表示できます。こんにちは、と名前の間に空白いれるためには、ダブルクオーテーションの中に空白を入れれば OK です。

問 2-7

input 関数を用いて、変数 a と変数 b に任意の文字列を代入し、その後 a と b の値を交換して表示するプログラムを作成せよ。

解答例（ソースコード）

```
a = input("変数 a に代入する文字列を入力してください：")
b = input("変数 b に代入する文字列を入力してください：")
tmp = a
a = b
b = tmp
print("変数 a: " + a)
print("変数 b: " + b)
```

解説

変数の中身を交換する方法を考える問題です。a=b, b=a と書いてしまうと、一回目の代入で元々の a の値が消えてしまいます。そのため、tmp = a のように別の変数に退避させておくことがポイントです。

また、講義資料中では扱っていない方法ですが、以下のように書いても変数の中身を交換できます。

```
a = input("変数 a に代入する文字列を入力してください：")
b = input("変数 b に代入する文字列を入力してください：")
a, b = b, a
print("変数 a: " + a)
print("変数 b: " + b)
```


Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 2-8

input 関数を用いて 1 つの整数を入力し、その一の位の数字のみを表示するプログラムを作成せよ。（ヒント：ある整数の一の位の値は、整数を１０で割った余りと等しい）

※この問題以降、特に変数名の指定がない場合は、変数名は任意に決めて良いものとする。

解答例（ソースコード）

```
n = input("数値を入力してください：")
print(int(n)%10)
```

解説

ある数で割った余り(剰余)は%を使えば求まります。input()を使って入力した値は str 型として生成されるので、演算前に int()で int 型にしないと%が使えません。

問 2-9

input 関数を用いて整数（今年の西暦）を入力し、「来年は（入力した西暦 + 1）年です。」と表示するプログラムを作成せよ。

解答例（ソースコード）

```
year = int(input("今年の西暦を入力してください："))
year = year + 1
print("来年は" + str(year) + "年です。")
```

解説

一問前と同じように、input()を使って入力した値を int 型にする必要があります。解答例のように input()を int()で囲っても OK です。最後に、文字列と来年の西暦を結合するので、year を str 型にします。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 2-10

input 関数を用いて 4 ケタの整数を 1 つ入力し、その各桁の数字を表示するプログラムを作成せよ。（ヒント：ある整数の一の位の値は、整数を１０で割った余りと等しい。十の位の値は整数を１００で割った余りを１０で割った商の整数部分と等しい。百の位の値は整数を１０００で割った余りを１００で割った商の整数部分に等しい。千の位の値は整数を１０００で割った商の整数部分に等しい）

解答例（ソースコード）

```
n = int(input("4 桁の数値を入力してください"))
one_place = n%10
ten_place = n%100 // 10
hun_place = n%1000 // 100
thou_place = n // 1000
print("1 の位は:" + str(one_place))
print("10 の位は:" + str(ten_place))
print("100 の位は:" + str(hun_place))
print("1000 の位は:" + str(thou_place))
```

解説

ヒントに書いてある通りに進めると解答のようなコードになります。商の整数部分を求めるときは/でなく//を使うことに注意してください。

第 3 部 コレクション

問 3-1

"バナナ"、"リンゴ"、"ミカン"、"イチゴ"を要素として持つリストを作成し、全ての要素を表示させた後、リストの 2 つ目の要素（この例だと”リンゴ”）を表示するプログラムを作成せよ。

注意：インデックスは 0 番から振られる。

解答例（ソースコード）

```
fruits = ["バナナ", "リンゴ", "ミカン", "イチゴ"]
print(fruits)
print(fruits[1])
```

解説

リストの作成方法は、変数名 = [要素, 要素,・・・]です。print(変数名)と書くと、リストの中の全ての要素を表示できます。特定の番号の要素にアクセスするには、インデックスを指定します。この問題の例だと、fruits[1]と書くことで、1 番目のインデックスの要素にアクセスすることができます。インデックスは 0 番から振られるので、先頭から数えて 2 つ目の要素にアクセスできます。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 3-2

"バナナ"、"リンゴ"、"ミカン"、"イチゴ"を要素として持つリストを作成し、全ての要素を表示させた後、1 番目のインデックスの要素をリストから削除し、新たに”ブドウ”という要素を追加し、再びすべての要素を表示するプログラムを作成せよ。

解答例（ソースコード）

```
fruits = ["バナナ", "リンゴ", "ミカン", "イチゴ"]
print(fruits)
fruits.pop(1)
fruits.append("ブドウ")
print(fruits)
```

解説

fruits.pop(1)と書くと、fruits というリストから 1 番目の要素であるリンゴを削除できます。

fruits.append(“ブドウ”)と書くと、fruits というリストの末尾にブドウという要素を追加できます。

問 3-3

key と value のペアが、"バナナ"：100、"リンゴ"：200、"ミカン"：80 の要素を持つ辞書を作成し、全ての要素を表示させた後、key として”ミカン”を指定して value を表示するプログラムを作成せよ。

解答例（ソースコード）

```
fruits = {"バナナ":100, "リンゴ":200, "ミカン":80}
print(fruits)
print(fruits["ミカン"])
```

解説

辞書の作成方法は、変数名 = [key 名:value,・・・]です。print(変数名)と書くと、辞書の中の全ての要素を表示できます。また、変数名[key]とすることで、key を指定して value を表示することができます。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 3-4

key と value のペアが、"バナナ"：100、"リンゴ"：200、"ミカン"：80 の要素を持つ辞書を作成し、全ての要素を表示させた後、key として”ミカン”を持つ要素を辞書から削除し、新たに”ブドウ”：300 を要素として辞書に追加し、再びすべての要素を表示するプログラムを作成せよ。

解答例（ソースコード）

```
fruits = {"バナナ":100, "リンゴ":200, "ミカン":80}
print(fruits)

fruits.pop("ミカン")
fruits["ブドウ"] = 300
print(fruits)
```

解説

fruits.pop("ミカン")と書くと、fruits という辞書からミカンとその value である 80 を削除できます。
fruits["ブドウ"] = 300 と書くと、fruits という辞書の末尾にブドウという要素を追加できます。

問 3-5

"バナナ"、"リンゴ"、"ミカン"、"イチゴ"を要素として持つリストを作成し、そのリストの長さを表示するプログラムを作成せよ。

解答例（ソースコード）

```
fruits = ["バナナ", "リンゴ", "ミカン", "イチゴ"]
print(len(fruits))
```

解説

len 関数を使うとリストや辞書の長さを得られます。また、len()の括弧の中に文字列を入れるとその文字列の長さを得られます。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 3-6

input 関数を用いて整数を 3 回入力し、それらを要素としたリストを作成せよ。その後、そのリストの要素の最大値と最小値と平均値を順番に表するプログラムを作成せよ。

解答例（ソースコード）

```
n1 = int(input("1 つ目の数値を入力してください : "))
n2 = int(input("2 つ目の数値を入力してください : "))
n3 = int(input("3 つ目の数値を入力してください : "))
numbers = [n1,n2,n3]
print(max(numbers))
print(min(numbers))
print(sum(numbers)/len(numbers))
```

解説

最大値と最小値は max と min 関数で得られます。平均値は合計値を sum 関数で求めた後、配列の要素数で割ると求まります。

問 3-7

input 関数を用いて整数を 2 回入力し、それらの整数をグループ A とする。次に、input 関数を用いて整数を 2 回入力し、それらの整数をグループ B とする。どちらかのグループに含まれる整数と、どちらのグループにも含まれる整数を以下の例のように表示するプログラムを作成せよ。

解答例（ソースコード）

```
a1 = int(input("グループ A の 1 つ目の数値を入力してください："))
a2 = int(input("グループ A の 2 つ目の数値を入力してください："))
b1 = int(input("グループ B の 1 つ目の数値を入力してください："))
b2 = int(input("グループ B の 2 つ目の数値を入力してください："))

a = {a1,a2}
b = {b1,b2}

print("どちらかのグループに含まれる値は以下の通りです。")
print(a|b)
print("どちらのグループにも含まれる値は以下の通りです。")
print(a&b)
```

解説

まず、入力されたそれぞれの値を要素とした set を作ります。|演算子で set の和集合が、&演算子で set の積集合が求まります。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

第 4 部 条件分岐

問 4-1

input 関数で整数(年齢)を入力し、年齢が 18 歳未満であれば、「未成年です」と表示するプログラムを作成せよ。

解答例（ソースコード）

```
age_str = input("年齢を入力してください:")
age = int(age_str)
if age < 18:
  print("未成年です")
```

解説

if 文の演習問題です。コロンの書き忘れやインデントのズレに注意してください。18 歳未満が条件なので、条件式を <= としてはいけません。

問4-2

input 関数で文字列を入力し、その文字列が"おはよう"に等しければ、「午前です」を表示するプログラムを作成せよ。

解答例（ソースコード）

```
greeting = input("あいさつしてください:")
if greeting == "おはよう":
  print("午前です")
```

解説

if 文の演習問題です。条件式では、文字列を比較することもできます。挨拶の表現はこれ以外にもありますが、これ以外の表現に対応したプログラムに書き換えるのも良い練習になると思います。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問4-3

input 関数で整数を入力し、その値が 1 ならば「天気は晴れです」、2 ならば「天気は曇りです」、それ以外ならば「天気は雨です」と表示するプログラムを作成せよ。

解答例（ソースコード）

```
weather = input("整数を入力してください:")
if weather == "1":
  print("天気は晴れです")
elif weather == "2":
  print("天気は曇りです")
else:
  print("天気は雨です")
```

解説

elif 節と else 節の演習問題です。elif 節は if 文とセットで記述します。elif 節の条件式の書き方は if 文と同様ですが、if 文やその elif 節より上の行に書かれている他の elif 文の条件式を満たさなかったときのみ、判定が行われます。else 節も if 文とセットで記述されます。else 節は他の if 文や elif 節の条件式を満たさなかったときのみ、処理が実行されます。

今回の演習では入力を整数としていますが、4-1 と異なり int 関数で整数に変換していません。それは、入力された値の大小を条件で比較していないためです。入力された値が特定の値であるかどうかは、文字列のままでも==を使えば判定できるので、整数に変換していません。ただし、文字列の 1 と等しいかどうか判定するので、判定式は、

```
if weather == "1":
```

のように 1 にダブルクオーテーションが付いていることに注意してください。

問4-4

input 関数で国語と数学のテストの点数を整数で入力し、数学の点が 90 点以上かつ国語の点が 80 点以上ならば「合格です」、それ以外ならば「不合格です」と表示するプログラムを作成せよ。

なお、数学 3000 点と国語 1000 点など 100 点より大きい整数を入力しても合格と表示しても構わない。(100 点より大きい整数の場合分けは不要)

解答例（ソースコード）

```
math_str = input("数学の点数を入力してください:")
math = int(math_str)
japanese_str = input("国語の点数を入力してください:")
japanese = int(japanese_str)

if math >= 90:
  if japanese >= 80:
    print("合格です")
  else:
    print("不合格です")
else:
  print("不合格です")
```

解説

ネストの演習問題です。if 文の中にさらに if 文を書くことで、より詳細な条件分岐ができます。数学が 90 点以上かつ国語が 80 点以上という条件はネストを使うと実現できます。ただし、一般的にはこのような条件式は以下のように and 演算子を使って実現することが多く、見た目にも分かりやすいです。

```
math_str = input("数学の点数を入力してください:")
math = int(math_str)
japanese_str = input("国語の点数を入力してください:")
japanese = int(japanese_str)

if math>=90 and japanese>=80:
  print("合格です")
else:
  print("不合格です")
```


Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問4-5

input 関数で西暦（整数）を入力し、それがうるう年である場合は「うるう年です」と表示し、そうでない場合は「うるう年ではありません」と表示するプログラムを作成せよ。

解答例（ソースコード）

```
year_str = input("西暦を入力してください：")
year = int(year_str)

if year % 400 == 0:
  print("うるう年です")
elif year % 100 == 0:
  print("うるう年ではありません")
elif year % 4 == 0:
  print("うるう年です")
else:
  print("うるう年ではありません")
```

解説

複雑な条件の if 文の演習問題です。実現方法はいろいろあるので、様々な値を入力して自分のプログラムが正しく動作しているか確認してください。400 で割り切れるかどうかという厳しい条件式を先に書くことで、ネストを使わずに書くことができます。もちろん、問題文に条件指定はありませんので、ネストを使って書いても正解です。ある数で割り切れるかは、%演算子を使ってあまりを計算し、それが 0 かどうかを判定すれば OK です。

問4-6

input 関数で 3 つの整数を入力し、最も大きい値を表示するプログラムを作成せよ。

解答例（ソースコード）

```
n1 = int(input("1 つ目の数値を入力してください : "))
n2 = int(input("2 つ目の数値を入力してください : "))
n3 = int(input("3 つ目の数値を入力してください : "))
max_val = n1
if max_val < n2:
  max_val = n2
if max_val < n3:
  max_val = n3
print("最大値は"+str(max_val))
```

解説

最大値を求める問題です。始めに n1 を最大値として設定し、その値より次の値が大きければ最大値を更新するという手法を取っています。条件分岐の章ですが、条件分岐を使わずに max 関数を使って以下のように解くこともできます。max 関数はリストだけでなく、要素を引数として羅列することでそれらの最大値を返すこともできます。

解答例（ソースコード）

```
n1 = int(input("1 つ目の数値を入力してください : "))
n2 = int(input("2 つ目の数値を入力してください : "))
n3 = int(input("3 つ目の数値を入力してください : "))
print("最大値は"+str(max(n1,n2,n3)))  # max([n1,n2,n3])としてもよい
```


Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 4-7

input 関数で整数を入力し、その値が 0 以上 100 以下の範囲外であれば「範囲外です」、70 以上ならば「合格です」、70 未満ならば「不合格です」と表示するプログラムを作成せよ。

解答例（ソースコード）

```
n = int(input("0～100 の整数を入力してください："))

if 0 <= n < 70:
  print("不合格です")
elif 70 <= n <= 100:
  print("合格です")
else:
  print("範囲外です")
```

解説

条件分岐のさせ方が色々と考えられる問題です。Python は 0<=n<70 という書き方ができるので、この条件式の書き方が読み手にとって分かりやすくお勧めです。

問4-8

input 関数で 3 つの文字列を入力する。その後 input 関数で 1 つの文字列を入力する。その文字列と最初の 3 つの内どれか 1 つ以上が同じ文字列なら「一致します」、そうでないなら「一致しません」と表示するプログラムを作成せよ。（例：りんご、メロン、すいか、りんごの順番で入力した倍は「一致します」と表示される。）

解答例（ソースコード）

```
s1 = input("1つ目の文字列を入力してください")
s2 = input("2つ目の文字列を入力してください")
s3 = input("3つ目の文字列を入力してください")
s4 = input("最後の文字列を入力してください")

if s4 == s1 or s4 == s2 or s4 == s3:
#  if s4 in [s1,s2,s3]:  でもよい
  print("一致します")
else:
  print("一致しません")
```

解説

複数の対象に対して同一判定をする問題です。順当に考えると s4==s1 という条件式を or 演算子で並べていくことになります。もちろん、それでも構いませんが、リストと in 演算子を使ってコメントアウトで記載した方法で判定しても OK です。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問4-9

10 個の適当な整数値を持つリストを作成し (例：numbers = [1,2,3,4,5,6,7,8,11,1])、このリスト内に重複した値がある場合は「重複しています」、ない場合は「重複していません」と表示するプログラムを作成せよ。なお、例で示したリストの場合必ず重複していますと表示されるはずなので、重複しないようにリストの中身を変更したケースでも動作を確かめること。

解答例（ソースコード）

```
numbers = [1,2,3,4,5,6,7,8,11,1]
if len(numbers) == len(set(numbers)):
  print("重複していません")
else:
  print("重複しています")
```

解説

リスト内の重複判定の問題です。色々なやり方がありますが、講義で扱った方法では set に変換して重複を削除する方法があります。重複がある場合は set に変換後の要素数と変換前の list の要素数が一致しません。これを利用して重複判定をします。

第 5 部 繰り返し

問 5-1

0 から 9 までの整数を順に表示するプログラムを while 文を利用して作成せよ。

解答例（ソースコード）

```
i = 0
while i < 10:
  print(i)
  i = i + 1
```

解説

while 文の演習問題です。while 文は条件式を満たしている間は、処理を繰り返します。条件式の書き方は if 文と同じです。for 文と異なり、条件式を満たさなくする処理を加えないと無限ループに陥るので気を付けましょう。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問5-2

"バナナ"、"リンゴ"、"ミカン"、"イチゴ"を要素として持つリストを作成した後、以下の実行例のように表示するプログラムを for 文を利用して作成せよ。

解答例（ソースコード）

```
fruits = ["バナナ", "リンゴ", "ミカン", "イチゴ"]
for fruit in fruits:
  print(fruit + "を食べます")
```

解説

for 文の演習問題です。for 文はイテラブルが持つ要素に対して、繰り返し同じ処理を行えます。ここでは、“を食べます”という文字列と結合して画面に表示しています。if 文や while 文と異なり、for の隣に書くのは、for 文の中で使う変数名：イテラブル名、となることに注意しましょう。

問 5-3

0 から 10 までの整数の内、偶数のみを順に表示するプログラムを for 文と range 関数を利用して作成せよ。

解答例（ソースコード）

```
for i in range(0,11,2):
  print(i)
```

解説

range 関数を用いた for 文の演習問題です。range 関数を使うことで等差数列を作れます。開始の数、終了の数、ステップを指定してオブジェクトを作成します。開始の数とステップは省略可能です。どのような動作になるか認識があいまいな人は再度資料を確認してください。ここでは、開始の値 0、終了の値 11、ステップ 2 を指定することで 0,2,4,6,8,10 という数列を作成しています。終了の値を 10 にすると数列に 10 が含まれなくなることに気を付けましょう。少し複雑になってしまいますが、if 文を使って以下のように書いても同様の表示が得られます。

解答例（ソースコード）

```
for i in range(11):
  if i%2 == 0:
    print(i)
```


Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 5-4

input 関数を用いて正の整数を 5 回入力し、その後、その中で最も大きい値のみを表示するプログラムを作成せよ。

解答例（ソースコード）

```
max_val = 0
for i in range(5):
  tmp_str = input("正の整数を入力してください:")
  tmp = int(tmp_str)
  if max_val < tmp:
    max_val = tmp
print(max_val)
```

解説

繰り返しと条件分岐の組み合わせの演習です。5 つの整数の内、最大となるものを探すアルゴリズムを考える問題です。今回は、最大となるものだけを最後に表示すればいいので、max_val というここまでの最大値と入力された値を比較し、より大きければ代入するという処理を、for 文の中で繰り返し実行しています。最初の値は比較するものがないので、あらかじめ max_val には 0 を代入しておきます。また、5 つの整数を全てリストに格納し、max(list)として計算しても構いません。

問5-5

0 が入力されるまで input 関数を用いて整数を繰り返し入力し、0 が入力された後、入力された全ての数値の合計値を表示するプログラムを作成せよ。

解答例（ソースコード）

```
total = 0
while True:
  num_str = input("整数を入力してください：")
  num = int(num_str)
  if num != 0:
    total = num + total
  elif num == 0:
    break
print(total)
```

解説

繰り返しと条件分岐の組み合わせの演習です。合計値を保存する変数を while 文の外側で定義しておき、while 文の内側で足していきます。while 文の条件式に True とだけ書いてありますが、こう書くと while 文は無限ループとなります。そのため、elif 節で break を書いてループを中断できるようにします。以下のように書けば、while True や break を使わなくても実現できます。

```
total = 0
num = 1
while num != 0:
  num_str = input("整数を入力してください：")
  num = int(num_str)
  if num != 0:
    total = num + total
print(total)
```


Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問5-6

西暦1900年～2000年までの、うるう年を全て表示するプログラムを作成せよ。

解答例（ソースコード）

```
for year in range(1900, 2001):
  if year % 400 == 0:
    print(year)
  elif year % 100 == 0:
    pass
  elif year % 4 == 0:
    print(year)
```

解説

繰り返しと条件分岐の組み合わせの演習です。4-5のソースコードを少し変更すると簡単に作れます。100で割り切れる場合の処理が今回は不要なので、passを記述しておきます。

問 5-7

FizzBuzz というゲームがある。このゲームを正しく行ったときのプレイヤーの発言を表示するプログラムを作成せよ。ただし、表示する発言は 1 から 40 までとする

解答例（ソースコード）

```
for i in range(1, 41):
  if i%3 == 0 and i%5 == 0:
    print("FizzBuzz")
  elif i%3 == 0:
    print("Fizz")
  elif i%5 == 0:
    print("Buzz")
  else:
    print(i)
```

解説

プログラミング入門で用いられる非常に有名な問題です。様々な解き方がありますが、解答例は繰り返しと条件分岐の組み合わせで解いています。15 の倍数を最初に処理するのがポイントです。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 5-8

数あてゲームのプログラムを作成せよ。要件は以下の通りである。

・「答」はあらかじめ 1～50 の整数の乱数として用意しておく。(乱数は以下の例のように書くことで生成できる。変数 rnd の中に 1~50 の整数が代入されている。import 文の意味は 6 章で学習予定)

・ユーザーが数字を入力すると、その数字が「答」よりも「大きい」か「小さい」かのみが表示される。

・ユーザーが正解した時点で「○回目で正解しました！」と正解までの入力回数を表示しプログラムを終了する。

解答例（ソースコード)

```
import random

rnd = random.randrange(50)+1
count = 1
while True:
  num = int(input("1～50 の整数を入力してください。"))
  if num == rnd:
    print(str(count) + "回目で正解しました。")
    break
  if num < rnd:
    print("答えよりも小さいです")
  else:
    print("答えよりも大きいです")
  count += 1
```

解説

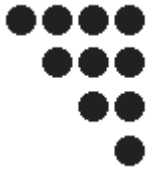
繰り返しと条件分岐、無限ループと break の演習問題です。無限ループではなく、while の継続条件を num != rnd としても OK です。この場合はそのほかの部分も少し変更が必要になりますのでチャレンジしてみてください。乱数の生成に import random という一文を使っていますが、import は 6 章で学習予定です。今のところは、乱数を生成するライブラリである random を使うための準備をしていると思っておいてください。randrange(50)の数値の部分を変えれば乱数の生成範囲を変えられます。

問5-9

下記の実行例のように、「●」を利用して三角形（長さ4）を表示するプログラムをそれぞれ作成せよ。ただし、一度にprintで出力できる●の数は一つとし、二重ループを使って表示すること。

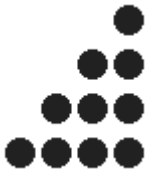
解答例（ソースコード）

```
for i in range(4):
  for j in range(4):
    if j - i >= 0:
      print("●", end="")
    else:
      print("  ", end="")
  print("")
```



解答例（ソースコード）

```
for i in range(4):
  for j in range(3,-1,-1):
    if j - i > 0:
      print("  ", end="")
    else:
      print("●", end="")
  print("")
```



解説

繰り返しと条件分岐の演習問題です。左側に黒丸が並んでいるパターンより一段階難しい問題です。行数を制御する外側のループと、列数（●や空白）を制御する内側のループを書きます。どちらの問題も1行ずれるごとに黒丸の数が1個ずつ増減しているので、黒丸の表示の制御には内側のループのカウンタjだけでなく、外側のループをカウンタであるiを使わなければならない、という発想ができればOKです。あとは具体的な値を入れて実行するなどして整理しながら正解の形に近づけていくといいでしょう。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

第 6 部 関数

問 6-1

以下の要件を満たすプログラムを作成せよ。

- 三角形の底辺と高さを引数とし、その面積を戻り値とする get_triangle_area 関数を定義する。
- input 関数を用いて底辺と高さを入力し、get_triangle_area 関数を用いて面積を計算し、計算結果を画面に表示する。

解答例（ソースコード）

```
def get_triangle_area(base, height):
  return base * height / 2

base_str = input("底辺を入力してください：")
teihen = int(base_str)
height_str = input("高さを入力してください：")
takasa = int(height_str)

area = get_triangle_area(teihen, takasa)
print(area)
```

解説

関数を定義してそれを利用する演習です。関数を定義するときに、関数名、引数、戻り値をどのように書くのかを講義資料で確認してください。引数に用いられている base と height はローカル変数なので、関数の外側では別の変数として同じ名前を利用できます。今回は違いを分かりやすくするため、あえて 5 行目で teihen、7 行目で takasa という変数名を付けていますが、これは base と height でも問題なく動作します。

問 6-2

以下の要件を満たすプログラムを作成せよ。

- 三角形の底辺と高さを引数とし、その面積を計算して画面に表示する print_triangle_area 関数を定義する。なお、戻り値はなしとする。
- input 関数を用いて底辺と高さを入力し、print_triangle_area 関数を用いて面積を画面に表示する。

解答例（ソースコード）

```
def print_triangle_area(base, height):
  area = base * height / 2
  print(area)

base_str = input("底辺を入力してください：")
base = int(base_str)
height_str = input("高さを入力してください：")
height = int(height_str)

print_triangle_area(base, height)
```

解説

関数を定義してそれを利用する演習です。6-1 と異なり関数の戻り値がありません。今回のように戻り値が不要な場合は定義する必要はありません。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 6-3

以下の要件を満たすプログラムを作成せよ。

- 自然数を引数とし、それが素数であれば True、素数でなければ False を戻り値とする check_prime 関数を定義する。なお、この関数への引数として 1 は想定しなくてもよい。
- input 関数を用いて自然数を入力し、check_prime 関数を用いて素数の判定をし、素数であれば「素数です」、素数でなければ「素数ではありません」と画面に出力する。

解答例（ソースコード）

```
def check_prime(num):
  for i in range(2, num):
    if num%i == 0:
      return False
  return True

num_str = input("2 以上の整数を入力してください：")
num = int(num_str)

if check_prime(num):
  print("素数です")
else:
  print("素数ではありません")
```

解説

総合的な演習です。素数判定の方法がポイントですが、ここでは 2 から与えられた引数まで全ての値で引数を割り、割り切れる値があるかをチェックしています。未学習の方法ですが、if 文の条件式にそのまま関数の呼び出しを書いています。これは、check_prime の戻り値が bool 型のオブジェクトなので可能な方法です。

問 6-4

以下の要件を満たすプログラムを作成せよ。

- 2 つの整数を引数とし、それらの数が小さい方を戻り値とする get_min 関数を定義する。
- 2 つの整数を引数とし、それらの最大公約数を戻り値とする get_gcd 関数を定義する。なお、この関数への引数として 1 以下の値は想定しなくてもよい。また、この関数の内部で get_min 関数を利用すること。
- input 関数を用いて 2 つの整数を入力し、get_gcd 関数を用いて最大公約数を計算し、計算結果を画面に出力する。

解答例（ソースコード）

```
def get_min(num1, num2):
    min_val = num2
    if num1 < num2:
        min_val = num1
    return min_val

def get_gcd(num1, num2):
    min_val = get_min(num1, num2)
    gcd = 1
    for i in range(2, min_val +1):
        if (num1%i == 0):
            if(num2%i == 0):
                gcd = i
    return gcd

if __name__ == "__main__":
  num_str = input("正の整数を入力してください：")
  num1 = int(num_str)
  num_str = input("正の整数を入力してください：")
  num2 = int(num_str)
  gcd = get_gcd(num1, num2)
  print(gcd)
```

解説

総合的な演習です。問題文の条件にあるように、自分で定義した関数は他の関数の中でも呼び出せます。最大公約数の計算方法がポイントですが、ここでは入力した 2 つの整数について割り切れる数を 1 から順番に探しています。最大公約数はどんなに大きくても 2 つの整数の内の小さい方の値にしかなりません。そのため、1 からその小さい方の値までの整数すべてについて、割り切れる数を探していけば最大公約数は見つかります。

if __name__ == "__main__":を付けると、そのブロックの中身はこのファイルを import されたときには実行されなくなります。後の問題(6-6)でこの関数を別の py ファイルから impoort するので、この段階でこのような実装にしています。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 6-5

input 関数を用いて 2 つの整数を入力し、math モジュールの gcd 関数を用いて最大公約数を計算し、計算結果を画面に出力せよ。

解答例（ソースコード）

```
import math


num1 = int(input("整数を入力してください。"))
num2 = int(input("整数を入力してください。"))
print("最大公約数は"+str(math.gcd(num1,num2))+"です。")
```

解説

標準ライブラリの使い方の演習です。6-4 で自作した最大公約数の判定アルゴリズムですが、Python に標準的にインストールされています。最大公約数を求める関数 gcd は math モジュールの中にあります。他にも便利な機能がたくさんあるので、自分で作る前にやりたいことを実現できるライブラリがないか調べるといいでしょう。もちろん自分で 1 から作るのもとても勉強になります。

問 6-6

input 関数を用いて 2 つの整数を入力し、問 6-4 で作成した get_gcd 関数を用いて最大公約数を計算し、計算結果を画面に出力せよ。今回の問題で使う py ファイル内に新たに get_gcd 関数を定義してはならず、問 6-4 で作成した関数を使うこと。また、必要であれば 6-4 のソースコードを変更すること。

解答例（ソースコード）

```
from ex6_4 import get_gcd


num1 = int(input("整数を入力してください。"))
num2 = int(input("整数を入力してください。"))
print("最大公約数は"+str(get_gcd(num1,num2))+"です。")
```

解説

外部ファイルの使い方の演習です。6-4 で自作した get_gcd 関数をインポートすることで、6-5 と同じくらい簡単に最大公約数を計算できます。6-4 のソースコードに注意が必要で、get_gcd 関数外の処理を if __name__ == "__main__":ブロックの中に入れておかないと今回の import 時にそれが実行されてしまいます。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問6-7

以下の仕様を満たすユーザとコンピュータが、じゃんけんを5回繰り返すプログラムを作成せよ。

・どちらかが3勝した時点で勝者を表示してプログラムは終了すること。

・あいこは1回とみなさず、決着がつくまで繰り返すこと。

・ユーザの入力はinput関数で与え、グー、チョキ、パーのいずれかとする。それらのいずれでもない入力がきた場合は再入力を促す。

・コンピュータの手はランダムで決定すること。

・各じゃんけんの勝負でユーザとコンピュータの手を表示してどちらが勝ったか明確にすること。

・コンピュータの手をランダムで決定するget_computer_choice関数と、ユーザとコンピュータの手を引数として勝負の結果を返す determine_winner関数をそれぞれ自分で定義して利用すること。

解説

総合演習です。ソースコードは長いので次のページに記載しています。ここまでで一番規模の大きいソースコードになりますが、一つ一つは学習済みの内容です。

get_computer_choice関数はコンピュータの手をランダムで決定するので、randomモジュールをimportして使いましょう。解答例ではrandom.choice(hands)として、handsリストの中からランダムに値を選択しています。学習済みの方法で実現するならreturn hands[random.randrange(3)]として0,1,2の乱数をインデックスとして手を選択するといいでしょう。

determine_winner関数は愚直に条件分岐を書いて勝敗判定すればOKです。判定条件が長くなりすぎるので、解答例では¥を使って条件式の途中に改行を挟んでいます。Python は条件式や文の途中に改行を挟むことはできませんが、¥を書くことで複数行にわたって一つの式や文を書けます。

あいこは決着がつくまで繰り返すので、ゲームの終了はどちらかが 3 勝することです。そのため、while 文の継続条件は user_wins < 3 and computer_wins < 3 として、どちらの勝ち数も3に満たないときにします。無限ループを作って、どちらかが3勝したらbreakするという書き方でもOKです。

ユーザの入力した値を not in ["グー", "パー", "チョキ"]でチェックすることで、これら 3 つの値であるか否かを調べています。not が付いているので、それら3つでないときにTrueになります。

解答例（ソースコード）

```
import random

def get_computer_choice():
  hands = ["グー", "チョキ", "パー"]
  return random.choice(hands)

def determine_winner(user_choice, computer_choice):
  if user_choice == computer_choice:
    return "draw"
  elif (user_choice == "グー" and computer_choice == "チョキ") or ¥
    (user_choice == "パー" and computer_choice == "グー") or ¥
    (user_choice == "チョキ" and computer_choice == "パー"):
    return "user"
  else:
    return "computer"

user_wins = 0
computer_wins = 0

while user_wins < 3 and computer_wins < 3:
  user_choice = input("出す手を選んでください。グー、チョキ、パー：")
  if user_choice not in ["グー", "パー", "チョキ"]:
    print("グー、チョキ、パーのいずれかで入力してください")
  else:
    computer_choice = get_computer_choice()
    print("じゃんけん、ぽん！")
    print("あなたの手: " + user_choice)
    print("コンピュータの手: " + computer_choice)

    result = determine_winner(user_choice, computer_choice)

    if result == "draw":
      print("あいこです")
    elif result == "user":
      user_wins += 1
      print("あなたの勝ちです。")
    else:
      computer_wins += 1
      print("コンピュータの勝ちです。")
    print("あなたの勝ち数：" + str(user_wins) + "  コンピュータの勝ち数：" + str(computer_wins))
if user_wins == 3:
  print("おめでとうございます。あなたの勝ちです。")
else:
  print("残念。また挑戦してください。")
```


Copyright Uchida Human Development , Co., Ltd. All rights reserved.

第 7 部 例外処理

問 7-1

input 関数を用いて 1 つの整数を入力し、その一の位の数字のみを表示するプログラムを作成せよ。（ヒント：ある整数の一の位の値は、整数を１０で割った余りと等しい）整数以外が入力された場合は、「整数以外が入力されました。」と代わりに表示すること。なお、この問題では負の値が入力される可能性は考慮しなくてよい。

解答例（ソースコード）

```
n = input("整数を入力してください：")
try:
  print(int(n)%10)
except ValueError:
  print("整数以外が入力されました。")
```

解説

例外処理の演習です。try ブロックの中に例外が起きそうな処理を、except ブロックの中に例外が起きた場合の処理を書きましょう。今回は str()の例外を処理するので、ValueError を指定します。解答例では 1 行だけですが、try ブロックの中は例外が起きそうな処理のみを記載するのが一般的です。例外に無関係な処理は、可読性が下がらない程度にブロック外に記載するようにしましょう。

問 7-2

0 が入力されるまで input 関数を用いて整数を繰り返し入力し、0 が入力された後、入力された全ての数値の合計値を表示するプログラムを作成せよ。整数以外が入力された場合は、「整数以外が入力されました。」と代わりに表示し、再度整数の入力を促すこと。

解答例（ソースコード）

```
total = 0
while True:
  try:
    num = int(input("整数を入力してください："))
  except ValueError:
    print("整数以外が入力されました。")
  else:
    if num != 0:
      total = num + total
    elif num == 0:
      break
print("合計は"+str(total)+"です。")
```

解説

例外処理の演習です。try except を while 文の中に書くのがポイントです。try ブロックの中に while 文を書くと、例外発生時にループの外に抜けてしまします。入力された整数についての処理は else ブロックの中に書くようにしましょう。解答例から else:を抜いた場合はどのような処理になってしまうかは考えて見てください。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 7-3

以下の仕様を満たすプログラムを作成せよ。

・プログラム中で任意の個数の整数を要素としたリスト、numbers を作成する。

・input 関数を用いて整数を入力し、その整数をインデックスとして numbers の要素を表示する。

・入力された値が整数でない場合は、「整数を入力してください」と表示する。

・入力された値が、numbers のインデックスの範囲外である場合は、「インデックスの範囲外です」と表示する。

・最後に「処理を終了します」と表示する。

・if 文は使用してはならない。（例外処理の練習のため）

解答例（ソースコード）

```
numbers = [17,39,62]
try:
  ind = int(input("表示したいリストのインデックスを入力してください："))
  num = str(numbers[ind])
except ValueError:
  print("整数を入力してください")
except IndexError:
  print("インデックスの範囲外です。")
else:
  print("指定したインデックスの値は"+num+"です。")
finally:  # この finally 節はなくても下の print 文は問題文の仕様通りに処理される。
  print("処理を終了します。")
```

解説

例外処理の演習です。2 種類の例外を処理するので、except を 2 つ書いています。リストの範囲外のインデックスを指定したときに発生する例外は IndexError です。最後の finally 節ですが、これはなくても問題の仕様通りに動作します。finally の有無で動作が変わる例は講義資料に記載してありますので、そちらも理解しておいてください。

第 8 部 文字列の操作

問 8-1

input 関数を用いて文字列を入力し、その文字列を表示するプログラムを作成せよ。ただし、入力と表示は abc を含む文字列が表示されるまで繰り返し、そのような文字列が入力された場合に繰り返しを終了すること。

解答例（ソースコード）

```
word = ""
while "abc" not in word:
  word = input("文字列を入力してください：")
  print("入力された文字列は" + word + "です。")
```

解説

文字列の検索の演習です。str オブジェクトの中身に特定の文字列が含まれるか否かを調べたいときは in 演算子が便利です。結果を反転させたい場合は not in を使いましょう。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 8-2

input 関数を用いてハイフン付きの電話番号を入力し、そのうちの市外局番のみを表示するプログラムを作成せよ。ただし、入力はハイフン付きの電話番号のみを考慮すればよい。

＊この問題において、市外局番とは 047-111-1111 や 03-1111-1111 のような電話番号の 047 や 03 の部分を指す。

解答例（ソースコード）

```
phone_number = input("電話番号をハイフン付きで入力してください。")
index = phone_number.find("-")
print("市外局番は" + phone_number[:index] + "です。")
```

解説

文字列の検索の演習です。含まれる文字列の位置（インデックス）を知りたいときは find メソッドを用いましょう。検索対象が複数ある場合は文字列の先頭にあるものが優先されるので、今回の演習には適しています。

文字列[a:b]と指定することで、文字列の a 文字目から b-1 文字目までを切り出すことができます。a を省略すると先頭から b-1 文字目までになります。

問 8-3

input 関数を用いて文字列を入力し、その文字列を先頭から 1 文字ずつ表示するプログラムを作成せよ。ただし、a が表示された場合はその時点で表示を終了すること。

解答例（ソースコード）

```
word = input("文字列を入力してください。")
for char in word:
  print(char)
  if char == "a":
    break
```

解説

文字列の操作の演習です。str オブジェクトはイテラブルなので、for x in y:の y の部分に使えます。このとき、文字列は 1 文字ずつに分解されて順番に x に代入されます。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 8-4

input 関数を用いて文字列を入力し、その文字列から「た」を抜いた文字列を表示するプログラムを作成せよ。

解答例（ソースコード）

```
word = input("文字列を入力してください。")
print(word.replace("た", ""))
```

解説

たぬきの演習です。replace メソッドを使うことで文字列に含まれる特定の文字列を置換できます。置換先を空文字""にすることで特定の文字列を削除することもできます。

問 8-5

We have a programming class today. #Python #Coding という文字列を変数に代入し、その文字列からハッシュタグを全て抽出して順番に表示するプログラムを作成せよ。この例では、

#Python

#Coding

と順番に表示されればよい。

解答例（ソースコード）

```
import re

word = "We have a programming class today. #Python #Coding"
pattern = "#¥w+"
hashtag = re.findall(pattern, word)
for w in hashtag:
  print(w)
```

解説

正規表現による文字列検索の演習です。正規表現を用いる際は re モジュールを import しましょう。今回はハッシュタグの検索で、 #Python と#Coding が抽出できれば OK です。そのため、#から始まる 1 文字以上の英数字を表す#¥w+を正規表現として用います。これを findall メソッドで検索すれば文字列に含まれる正規表現のパターンを全て抽出できます。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

問 8-6

We The incidents occurred on 1910-01-12 and 2012-12-23. The case number for that incident is 31522-11-21113. という文字列を変数に代入し、その文字列から日付を全て抽出して順番に表示するプログラムを作成せよ。この例では、

1910-01-12

2012-12-23

と順番に表示されればよい

解答例（ソースコード）

```
import re

word = "The incidents occurred on 1910-01-12 and 2012-12-23. The case number for that incident is 31522-11-21113."
pattern = "¥s¥d{4}-¥d{2}-¥d{2}"
date = re.findall(pattern, word)

for w in date:
  print(w)
```

解説

正規表現による文字列検索の演習です。ハイフンを挟んだ 4 桁、2 桁、2 桁の数字を抽出します。これを抽出するには¥d{4}-¥d{2}-¥d{2}という正規表現を用いますが、これでは 31522-11-21113 の中から 1522-11-21 が抽出されてしまいます。そのため、¥s¥d{4}-¥d{2}-¥d{2}のように先頭に空白文字を含む正規表現とすれば日付のみを抽出できます。（厳密には空白文字も含まれてしまいますが、気になる場合は表示の際に w[1:]として空白文字以降を表示すれば OK です。）

問8-7

input 関数を用いて文字列を入力し、その文字列から連続する空白文字を単一の半角スペースに置き換えた文字列を表示するプログラムを作成せよ。例えば、This is　　a　　pen.と入力された場合、表示される文字列はThis is a pen.となる。

解答例（ソースコード）

```
import re
word = input("文字列を入力してください。")
print(re.sub("¥s+", " ", word))
```

解説

正規表現による文字列置換の演習です。正規表現を用いた文字列置換には sub を用います。引数が 3 つと多いので順番を間違えないようにしましょう。re.sub(マッチングパターン、変換後の文字列、変換対象の文字列)です。今回は、マッチングパターンを取りたい文字が空白文字なので”¥s+”を使っていますが、半角スペースの連続だけでいいなら” +”としてもマッチング可能です。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.